

Maximum Margin Classifier

© Copyright 2021 Fusemachines, Inc. | All rights reserved

Prerequisites

Students must know of the following topics:

- Concept of KNN, ML level 1 and 2
- Concept of Decision tree

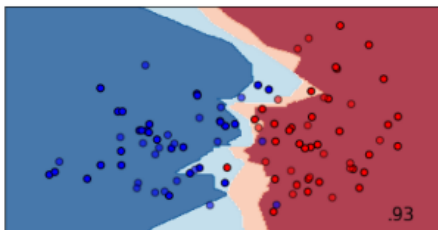
Learning Objectives

After reading this notebook, students will be able to:

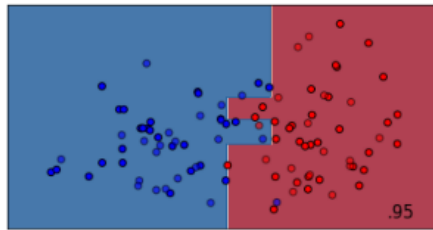
- Compare different type of decision boundary from different algorithms.
- Understand optimum hyperplane, margin and support vectors.
- Explain the maximum margin hyperplane and its mathematical derivation.
- Optimize SVM (maximum margin classifier) using cvxopt python library.

Recall: Decision Boundary of Different Algorithms

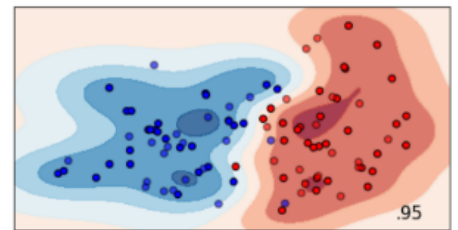
The main aim of supervised learning is to correctly identify the learning procedure that can classify previously unseen data patterns. We have already seen the different classifier types in the previous chapter that can linearly separate the data points. Let's recall how KNN, Decision tree creates a decision boundary to separate the data points.



Fig(a): KNN



Fig(b): Decision Tree



Fig(c): SVM

Figure 1: Decision Boundary of different algorithm

The decision boundary of a decision tree is determined by overlapping orthogonal half-planes (representing the result of each subsequent decision)

In KNN, we use k-fold cross-validation to determine which of our models is best. In k-fold-cross-validation, we compute the mean of the metric of each fold. Thus we separate the data points.

Whereas in SVM, to separate data points, we choose the hyperplane, which has the largest margin among all the other hyperplane. Don't worry about how the SVM creates a decision boundary with these terms. We will go one by one.

We will start with motivation. We know that the line is used to separate between the two groups of data. The figure below shows the three separating lines A, B, and C. The line A is okay, though it separates the line and has the training error zero. But may arise the problem during testing the new data as it lies close to the negative

sample. So does the line C as close to the positive examples. In contrast to line B, it would perform okay in the case of new testing data.

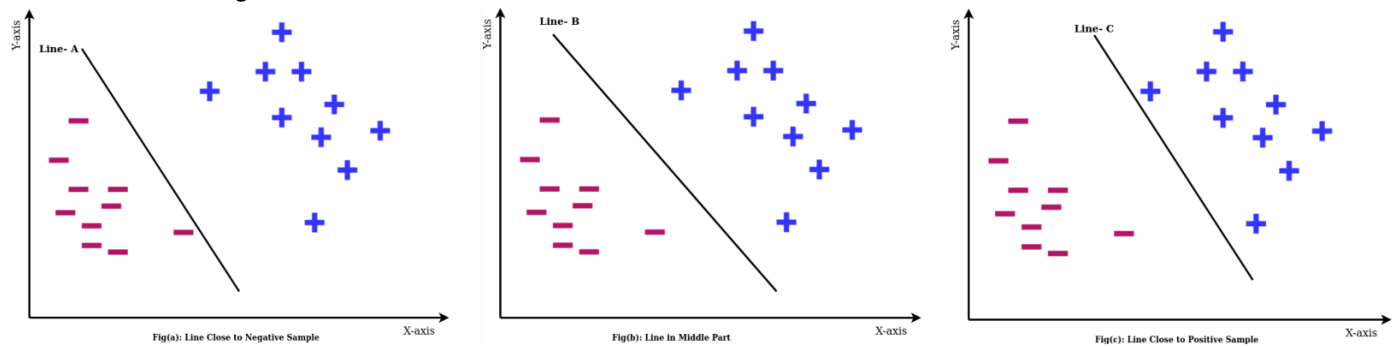


Figure 2: Three different Hyperplane separating for the same data points.

This is the whole idea of the maximum margin classifier in SVM.

Alright! For SVM, You may choose some line(hyperplane) to separate the two different classes. Let's understand the working mechanism of SVM with small problem formulation.

Problem: Suppose we have a binary data of negative and positive samples in the plane.

Goal: To separate these data into two groups using the line.

Strategy: To find the street to separate these data points. The street should be widest as possible. The widest street helps to make our decision. We will see all the derivations onto this.

Intuition: let's consider the points near the street as gutters. The gutters on the side of the street might help to define the working mechanism of SVM.

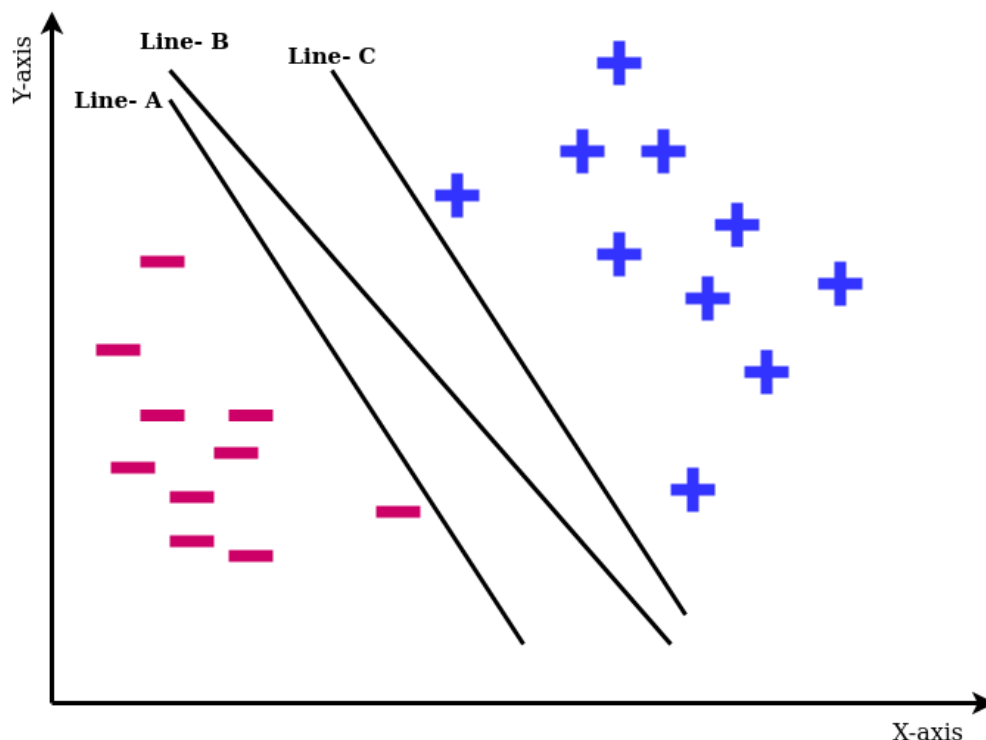


Figure 3: Three different Hyperplane separating data points in single figure

The figure above shows that there are many lines (hyperplane) to separate the given data points. Among A, B, C hyperplane, which one do you think would be better? How can we identify the best hyperplane? Don't worry; it's not as hard as you think!

Let's recap the geometric view of the line, as shown in the figure below. Mathematically,

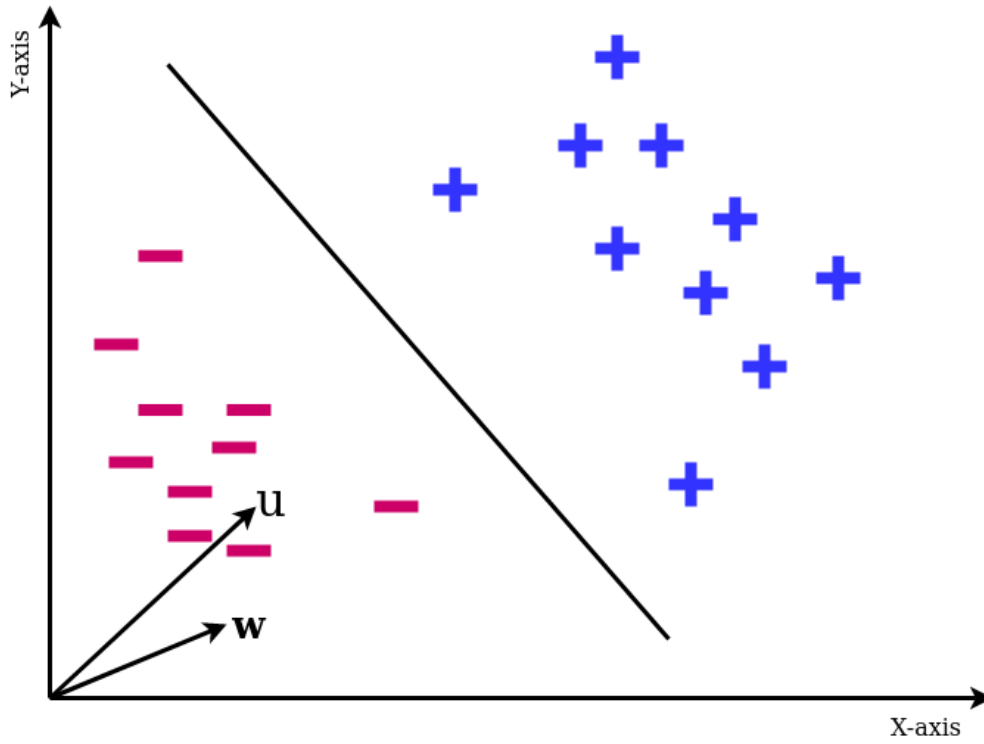


Figure 4: w and u vector in binary data points

As we have the sample data in a space. Let's imagine a vector w of any length and constrained to be perpendicular to the median line. Also, the unknown vector u , which we would like to classify. That means whether the unknown sample is positive or negative.

As we know, w can be scaled further of any length. Now, we will project the vector u onto w . We will do the dot product of w and u to find the scalar projection of u onto w .

$$w \cdot u$$

Now what we can do is the dot product of w and u and measure whether it is equal to or greater to some constant c . If it is greater or equal to the constant, then it crosses the line that is unknown point u is a positive sample. Else the negative sample.

$$w^T \cdot u \geq c \rightarrow u_+ \\ \text{Else, } u_-$$

Also,

$$w^T \cdot u \geq c \rightarrow u_+$$

These two equations are the same as the value of constant c is equal to $-b$

We will define a linear classifier for binary classification problem. Here, x represent as a features and y as labels with the values 1 or -1. then,

$$\begin{aligned} wx_+ + b &\geq 1, & \text{Positive class} \\ wx_- + b &\leq 1, & \text{Negative class} \end{aligned}$$

Collectively,

$$\begin{aligned} y_i(wx_i + b) &\geq 1 \\ y_i(wx_i + b) - 1 &\geq 0 \end{aligned}$$

For x_i observations in gutter

$$y_i(wx_i + b) - 1 = 0 \quad (2)$$

Note:

- we will be using $\{-1,1\}$ as a label values instead of $\{0,1\}$

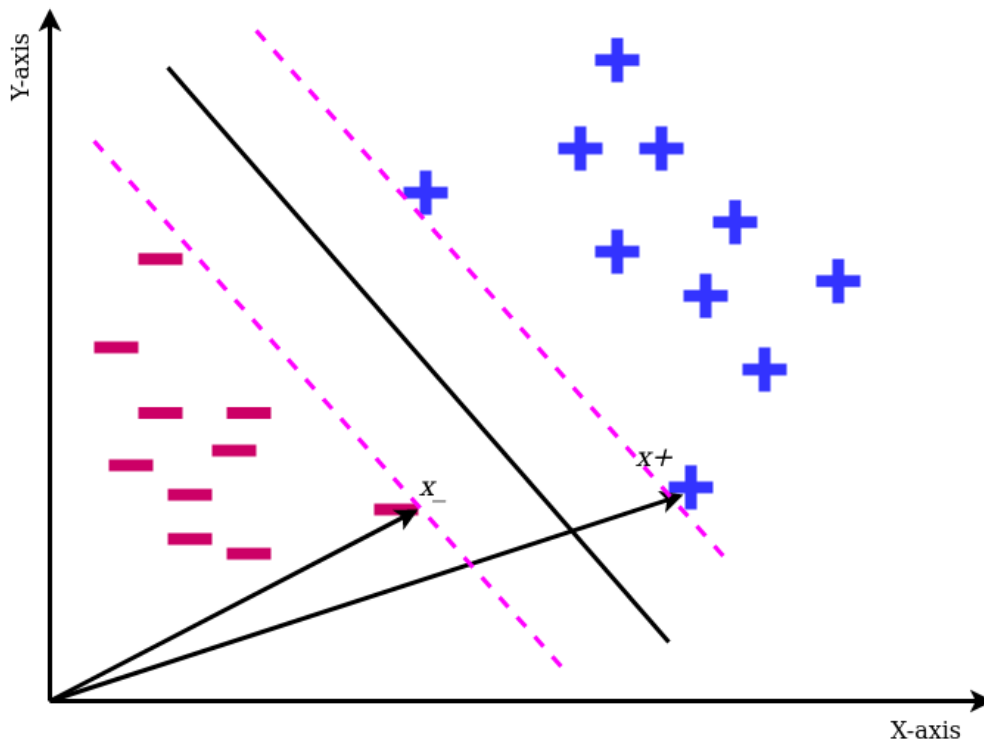


Figure 5: Representation of gutter vector(x_+ and x_-)

The line (hyperplane) is defined by a vector $\mathbf{w}$ and its equation is,

$$\mathbf{w}^T \mathbf{x} + b = 0 \quad (1)$$

Now, Imagine we have two gutter vectors representing x_+ and x_- as shown in the above figure. To find the width of the street, let's consider their difference vector as $(x_+ - x_-)$. Now the dot product of the difference vector to the unit vector in the direction of the median i.e $\frac{\mathbf{w}}{||\mathbf{w}||}$ will give the width of the street.

$$\begin{aligned} width &= (x_+ - x_-) \frac{\mathbf{w}}{||\mathbf{w}||} \\ &= x_+ \frac{\mathbf{w}}{||\mathbf{w}||} - x_- \frac{\mathbf{w}}{||\mathbf{w}||} \\ &= \frac{\mathbf{w}x_+ - \mathbf{w}x_-}{||\mathbf{w}||} \end{aligned} \quad (3)$$

Now, in the equation(2);

$$y_i(\mathbf{w}x_i + b) = 1$$

If Positive then,

$$1(\mathbf{w}x_+ + b) = 1$$

$$\text{or,} \quad \mathbf{w}x_+ + b = 1$$

$$\mathbf{w}x_+ = 1 - b \quad (4)$$

If Negative then,

$$-1(\mathbf{w}x_- + b) = 1$$

$$\text{or, } -\mathbf{w}x_- - b = 1 \quad \mathbf{w}x_- = -(1 + b) \quad (5)$$

Substituting the value of equation(4) and (5) in equation(3). We get,

$$\begin{aligned} \text{width} &= \frac{(1 - b) + (1 + b)}{\|\mathbf{w}\|} \\ &= \frac{2}{\|\mathbf{w}\|} \end{aligned} \quad (6)$$

Equation (6) reflects the widest street.

Now, our objective is to find a hyperplane that maximizes the width. We can write the equation (6) as,

$$\begin{aligned} &= \max \frac{2}{\|\mathbf{w}\|} \\ &= \max \frac{1}{\|\mathbf{w}\|} \end{aligned}$$

But just maximizing would not provide us the required hyperplane. Because, the hyperplane maximized can lie anywhere in the space without actually separating the two classes. For this reason, we are going to introduce a constraint in our optimization problem and use constraint optimization methods to find the optimal hyperplane. The constraint is that the hyperplane should lie in between the data points of two classes. With this constraint, the optimal hyperplane would have the maximum margin and lie in-between the two classes separating them.

As we mentioned earlier that $y \in \{1, -1\}$ are the two class labels, then let's suppose that all the points on the negative side ($\mathbf{w}^T \mathbf{x} + b < 0$) of the hyperplane belong to the negative class ($y = -1$) and, all the points on the positive side ($\mathbf{w}^T \mathbf{x} + b > 0$) of the hyperplane belongs to the positive class ($y = 1$). A negative side of the hyperplane is one of the two sides where all the points of that side yields a negative value from the equation of hyperplane ($\mathbf{w}^T \mathbf{x} + b$) and vice-versa. Then, for a hyperplane defined with parameter $\mathbf{W}$ and b ,

$$y_i \cdot (\mathbf{w}^T \mathbf{x}_i + b) \geq 0$$

Now the optimization criterion becomes

$$\begin{aligned} &\max \frac{1}{\|\mathbf{w}\|_2} \\ &\text{subject to } y_i (\mathbf{w}^T \mathbf{x}_i + b) \geq 0 \end{aligned}$$

Also we can write as,

$$\begin{aligned} &\max \frac{1}{\|\mathbf{w}\|_2} \\ &\Rightarrow \min \|\mathbf{w}\| \\ &\Rightarrow \min \frac{1}{2} \|\mathbf{w}\|^2 \\ &\text{subject to;} \\ &y_i (\mathbf{w}^T \mathbf{x}_i + b) \geq 1 \end{aligned} \quad (7)$$

Solving the equation (7) for the value of $\mathbf{w}$ gives us the solution for the optimal hyperplane of the support vector machine.

By the definition, The margin in the SVM is the distance of the hyperplane to the nearest data points.

Visualization of Optimal hyperplane using cvxopt library

We are going to find an optimal hyperplane by maximizing the equation (7) using a convex optimization library. You can refer to [this \(https://courses.csail.mit.edu/6.867/wiki/images/a/a7/Qp-cvxopt.pdf\)](https://courses.csail.mit.edu/6.867/wiki/images/a/a7/Qp-cvxopt.pdf) document to learn in detail.

In []:

```
# import the library
import cvxopt

# import solvers module that contains quadratic solver
from cvxopt import solvers

# all the data should be converted to matrix object
# can be thought it being equivalent to numpy array
from cvxopt import matrix
import numpy as np

import matplotlib.pyplot as plt
```

In []:

```
# Input data points
X = np.array([[0,0], [0, 1], [1, 0], [1, 1], [2, 0], [0, 2]])

# Input points are extended by adding 1 at the beginning for bias term
# so [5,2] point becomes [1, 5, 2]
X_ex = np.hstack((np.ones((X.shape[0], 1)), X))
print(X_ex.shape)
#nf represent the number of features
nf = 2

# Class label for individual data points
y = np.array([-1, -1, -1, 1, 1, 1])

y = np.reshape(y, (y.shape[0], 1))

# convert all data to matrix object
# these matrix are calculated as explained above
G = matrix(y * -X_ex, tc='d')
h = matrix(-np.ones((X.shape[0], 1)), tc='d')
P = matrix(np.identity(nf + 1), tc='d')
q = matrix(np.zeros(nf + 1), tc='d')
```

(6, 3)

In []:

```
# call the quadratic solver function with the appropriate parameters
sol = solvers.qp(P, q, G, h)
```

	pcost	dcost	gap	pres	dres
0:	5.6250e-01	3.1042e+00	4e+00	2e+00	2e-15
1:	1.4693e+00	5.1033e+00	3e-01	6e-01	6e-16
2:	9.2454e+00	7.7766e+00	1e+00	5e-16	2e-13
3:	8.5076e+00	8.4879e+00	2e-02	4e-16	1e-14
4:	8.5001e+00	8.4999e+00	2e-04	2e-16	1e-14
5:	8.5000e+00	8.5000e+00	2e-06	3e-16	1e-14

Optimal solution found.

In []:

```
# let's get the value of theta from the solution
theta = np.array(sol['x'])

# get a list of values for x (first feature)
_x = np.arange(-1, 3, 0.2)

# calculate values of y (next feature) using parameter theta and x

# this is from equation of hyperplane
_y = - (_x * theta[1] + theta[0])/theta[2]

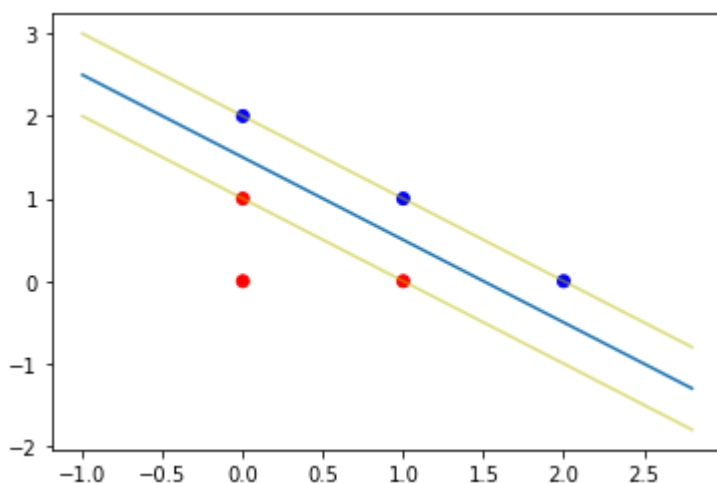
# this is from the equation of margin at positive side
_yp = (1 - _x * theta[1] - theta[0])/theta[2]

# this is from the equation of margin at negative side
_yn = (-1 - _x * theta[1] - theta[0])/theta[2]

plt.scatter(X[:, 0], X[:, 1], c=['r' if _y < 1 else 'b' for _y in y])
plt.plot(_x, _y)
plt.plot(_x, _yp, c='y', alpha=0.5)
plt.plot(_x, _yn, c='y', alpha=0.5)
```

Out[5]:

[<matplotlib.lines.Line2D at 0x7f8df5b0de10>]



In the plot above, the middle line is the optimal hyperplane. Two lines parallel to the optimal hyperplanes on the two side of the above plot represents the margin. The points that lie on these two parallel lines are the support vectors.

Key Takeaways

- Hyperplane is a decision boundary that helps to classify, separate, differentiate between different groups of data.
- The margin in the SVM is the distance of the hyperplane to the nearest data points.
- Hyperplane with low margin results the high chance of misclassification.
- Support vectors are the data points that are closest to the separating hyperplane.

References

- Books
 - Nello Cristianini, John Shawe-Taylor(2000), An Introduction to Support Vector Machines and other Kernel-based Learning Methods, 1st edition
 - Check unit 6.1.1, page 94 to understand the Maximal Margin Classifier.
 - Stephen Marsland, Machine Learning An Algorithmic Perspective, 2nd edition
 - Check Chapter 8, page 170 to understand the optimal separation and its constrained.